

IntellMeet

AI-Powered Enterprise Meeting & Collaboration Platform

Day 1 to Day 3 Development Documentation

This document summarizes the eight completed development phases from Day 1 through Day 3, covering project setup, backend and database, authentication, meeting management, real-time communication, video conferencing, collaboration, and live transcription.

Technology Stack: React.js, Vite, Axios, React Router, Node.js, Express.js, MongoDB, Mongoose, JWT, bcrypt, Socket.io, WebRTC, Browser Speech Recognition, Visual Studio Code.

DAY 1 — Project Foundation and Authentication

Objective: Establish the application architecture, backend, database, and secure authentication.

Phase 1 — Project Setup

Work Completed: Set up separate React/Vite frontend and Node.js/Express backend applications and connect them through APIs.

Technologies Used: React, Vite, Node.js, Express.js, Axios, React Router

Outcome: Established the full-stack development environment.

Phase 2 — Backend and Database

Work Completed: Created the backend infrastructure and connected MongoDB using Mongoose. Structures were established for users, meetings, messages, transcripts, summaries, action items, and workspaces.

Technologies Used: Node.js, Express.js, MongoDB, Mongoose

Outcome: Enabled persistent storage and retrieval of core application data.

Phase 3 — Authentication and Authorization

Work Completed: Implemented registration and login with bcrypt password hashing and JWT-based authentication for protected API access.

Technologies Used: bcrypt, JWT, Express middleware

Outcome: Users can securely authenticate and access protected functionality.

DAY 2 — Meeting and Real-Time Communication

Objective: Develop the core meeting system and real-time communication between participants.

Phase 4 — Meeting Management

Work Completed: Implemented meeting creation, unique room IDs, participant management, meeting retrieval, cancellation, and meeting status handling.

Technologies Used: React, Express.js, MongoDB, Mongoose

Outcome: Established the core meeting-management system.

Phase 5 — Real-Time Communication

Work Completed: Integrated Socket.io for meeting rooms, participant presence, WebRTC signaling, chat, transcript updates, and disconnect events.

Technologies Used: Socket.io, Node.js

Outcome: Enabled real-time communication and event exchange.

Phase 6 — Video Conferencing

Work Completed: Implemented WebRTC camera and microphone communication, using Socket.io for offers, answers, and ICE candidates.

Technologies Used: WebRTC, Socket.io, browser media APIs

Outcome: Enabled live audio and video communication.

DAY 3 — Collaboration and Live Transcription

Objective: Enhance the meeting room with collaboration tools and capture meeting conversations as text.

Phase 7 — Collaboration Features

Work Completed: Added real-time chat, participant management, meeting controls, and screen sharing. Chat messages are exchanged through Socket.io and stored in MongoDB.

Technologies Used: Socket.io, WebRTC, browser media APIs, MongoDB

Outcome: Expanded the meeting room into a collaborative workspace.

Phase 8 — Live Transcription

Work Completed: Integrated browser speech recognition to convert spoken meeting conversations into text. Transcript records include meeting room, speaker, text, user, and timestamp and are stored in MongoDB.

Technologies Used: Browser Speech Recognition, Socket.io, MongoDB

Outcome: Meeting conversations can be captured and stored for later processing.

Overall Day 1–3 Summary

Day	Focus	Completed Work
Day 1	Foundation	Project setup, backend, database, authentication
Day 2	Meeting System	Meeting management, Socket.io, WebRTC
Day 3	Collaboration	Chat, screen sharing, live transcription

Status after Day 3: Eight development phases are completed. The platform currently includes the full project foundation, secure authentication, meeting management, real-time communication, WebRTC video/audio, chat, screen sharing, participant controls, and live transcription.